A Cognitive Framework for Engineering Growth

From Remembering to Creating

A level-guided journey for software engineers.

The difference between a senior engineer and a staff engineer isn't what they ship — it's how they think about what to ship.

THE QUESTION THIS FRAMEWORK ANSWERS

Growth is a change in thinking, not title.

Six cognitive verbs map the questions an engineer learns to ask — your level is the highest verb you reliably own, and how far that thinking travels.

01	**Remember**	Recall facts, syntax, rules.	How is this done?
02	**Understand**	Explain behavior and dependencies.	Why does it work?
03	**Apply**	Use patterns in new contexts.	How do I make it work here?
04	**Analyze**	Decompose across boundaries.	Where does it break?
05	**Evaluate**	Judge options against strategy.	Which option should win?
06	**Create**	Synthesize what isn't there.	What should exist?

LEVEL 1 · REMEMBERING

Execute with accuracy.

How is this done?

You recall syntax, patterns, and standards — the path is known, and the job is to follow it without error.

Level 1 · split-panel proof

YOU KNOW YOU'RE HERE WHEN

You implement a feature via an existing API. Tests pass, review comes back clean, and rework is rare.

FOLLOWS EXAMPLES WELL

Compiles, runs, and tests locally with confidence.

WORKS FROM BRIEF TICKETS

Consistent quality without hand-holding.

LEVEL 2 · UNDERSTANDING

Explain the why behind the what.

Why does it work?

You read the system, trace the flow, and explain failure modes — changes you make no longer surprise anyone at integration time.

Level 2 · split-panel proof

YOU KNOW YOU'RE HERE WHEN

You trace a request end-to-end, pinpoint a bottleneck, and explain the tradeoff between two fixes to a peer.

DRAWS THE FLOW

Reads code you didn't write and explains failure modes clearly.

DOCUMENTS ASSUMPTIONS

Proposes a small improvement and ships it safely.

LEVEL 3 · APPLYING

Own the outcome end-to-end.

How do I make it work here?

You use known patterns in new contexts, shipping features with reliability, observability, and on-call readiness.

Level 3 · split-panel proof

YOU KNOW YOU'RE HERE WHEN

You design and deliver a scalable microservice with CI/CD, SLOs, and runbooks — and you carry the pager.

DELIVERS END-TO-END

Meets SLOs, on-call ready and effective.

ADAPTS THE PATTERN

Fits a standard pattern to a novel domain constraint.

LEVEL 4 · ANALYZING

See the system, not just the service.

Where does it break?

You decompose complexity across boundaries — when something breaks between services, you find the root cause.

Level 4 · split-panel proof

YOU KNOW YOU'RE HERE WHEN

You re-architect a data pipeline to resolve distributed latency that three teams couldn't isolate independently.

FINDS ROOT CAUSES

Traces across services and proposes options with tradeoffs.

LEADS THE CHANGE

Reduces P95 latency or infra cost measurably.

LEVEL 5 · EVALUATING

Judge options against strategy and time.

Which option should win?

You frame criteria, compare architectures, and make calls that hold up over a three-year horizon.

Level 5 · split-panel proof

YOU KNOW YOU'RE HERE WHEN

You choose a modular monolith over microservices — weighing scale, TCO, and compliance — and get buy-in from four teams.

FRAMES CRITERIA

Compares options and recommends with clear rationale.

GAINS ADOPTION

Aligns multiple teams on the decision and its tradeoffs.

LEVEL 6 · CREATING

Build what didn't exist before.

What should exist?

You synthesize new frameworks, platforms, and operating models — the artifact you produce becomes the standard others build on.

Level 6 · split-panel capstone

THE SIGNAL

You design a cross-cloud, policy-aware data platform framework, and teams across the enterprise adopt it as their foundation.

REFERENCE ARCHITECTURE

The implementation exists and is validated.

ORGANIZATION-WIDE ADOPTION

Measurable outcomes follow.

The second axis: how far it reaches.

The verb is one axis — how you think. **Reach** is the other — how far what you make travels. Your level is where they meet: **Evaluate** for your team and **Evaluate** across the org are different levels with the same verb.

I

Own the verb

You can do the cognitive work — correct, clear, complete. So far it reaches only you.

→

II

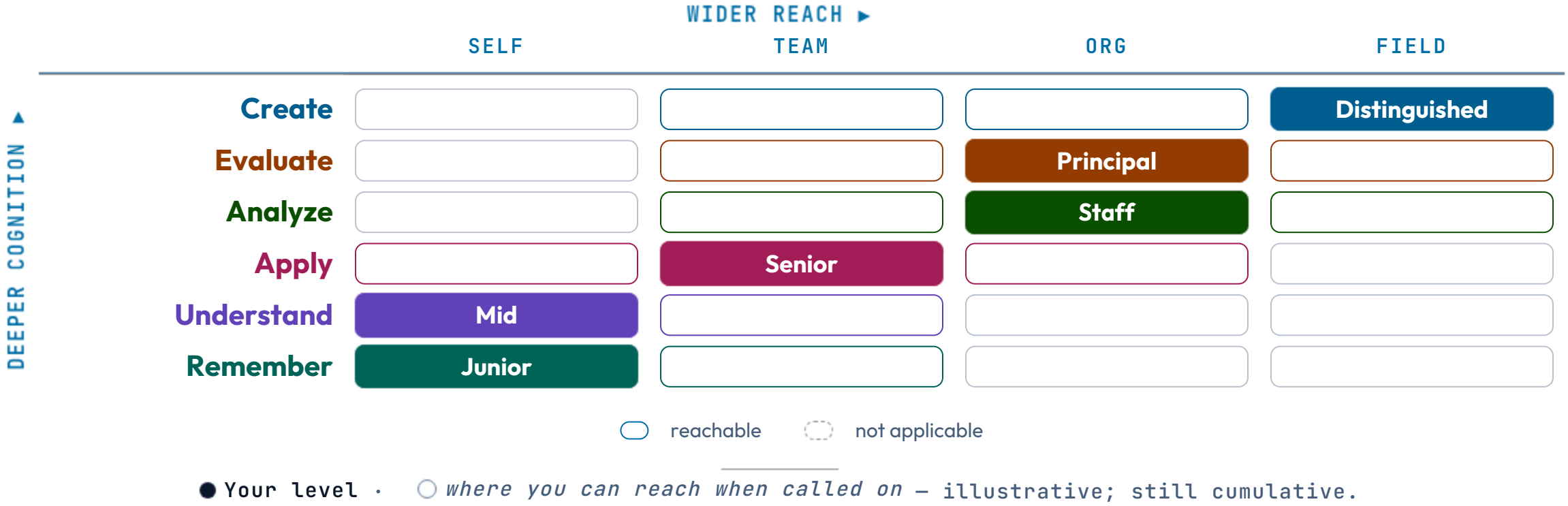
Widen the reach

The work travels: team, org, field. Documented, adopted, durable — the farther it carries, the higher the level.

✦ *Most engineers stall here — not on the thinking, but on making it travel past their own hands. That gap, not the verb, is what earns the title.*

Your level is a cell, not a rung.

Your title is the diagonal — the same verb at a wider reach is a different level.



● Your level · ○ *where you can reach when called on* — illustrative; still cumulative.

How to use this tomorrow.

STEP 01

Place yourself

Name the verb you own and how far it reaches today. Be honest — most of us straddle two.

STEP 02

Pick your next move

A deeper verb, or the same verb carried wider — concrete, time-bound, tied to real work.

STEP 03

Collect the evidence

Design docs, ADRs, before/after metrics, postmortems — artifacts that prove the shift happened.

Grow on two axes, not one.

Deepen the verb — change the question you ask. Widen the reach — make the work travel past your own hands. Every level you've passed still lives in how you work; the title follows when both axes show up in the evidence.